SIIT Study Hub

Database Design & Procedures — ELI5 Q&A Cheat Sheet

CSS326 Database Programming Laboratory — Week 7 Proposal

Touch-ing Yinyong

Quick-reference for live presentation Q&A

How to use this. These are plain-English answers to the "why did you do it this way" questions an audience is most likely to ask. Say the ELI5 line first. Only reach for the second line if someone pushes back or asks a follow-up.

1 Security

"Why salt AND hash the password?"

ELI5: Hashing scrambles the password into gibberish that can't be reversed — even we can't see it. But if two students both use "password123", plain hashing gives them the *identical* gibberish, so an attacker with a precomputed lookup table (a rainbow table) cracks both instantly. Salt is a random extra ingredient mixed in per user before hashing, so the same password produces different gibberish for everyone. Hash scrambles it, salt makes every scramble unique.

If pushed further: The salt is stored per-row in `VARBINARY`, and only `sp_register_user` is allowed to set `password_hash` — so the raw password never touches application code or logs.

---

2 Why so much "advanced" design already?

"Why so many advanced features (procedures, triggers, transactions) right now, at proposal stage?"

ELI5: We didn't add these to look impressive — each one is the direct fix for a specific, real problem, and each one lines up with exactly the week CSS326 teaches that concept.

If pushed further: We're not building ahead of the syllabus, we mapped our design onto it: procedures land Week 10, triggers Week 11. Right now we're showing we understand *why* each piece is needed before we write a line of SQL.

---

3 Schema Design

"What is CourseOffering, and why do we need it?"

ELI5: "CSS326" is the course. But CSS326 gets taught over and over — different semester, sometimes a different instructor. CourseOffering is one specific instance of that: "CSS326, taught by Dr. A, in 2026/1." Without it, every submission would have to re-type the instructor and semester as plain text, and fixing one typo wouldn't fix the other fifty rows that have it too.

If pushed further: It's the difference between writing the teacher's name on every homework page versus writing it once on the syllabus and pointing to it.

"What level of normalization is this?"

ELI5: Third normal form (3NF) — every non-key column depends on the whole primary key and nothing else.

If pushed further: The clearest example is CourseOffering itself: in an earlier prototype, instructor and semester were transitively dependent on the course but stored as duplicated free text anyway. Pulling them into their own table with their own key is the textbook 3NF fix. (1NF is satisfied because every field holds one atomic value, not a comma-jammed list; 2NF isn't really at stake since we don't use composite primary keys.)

"Why split Content into ExamIntel and Materials instead of one big table?"

ELI5: If everything lived in one table, every row would need columns for both "exam duration" *and* "markdown notes," half of them blank depending on the type. Content holds what's shared — who posted it, which course, the attachment. ExamIntel and Materials each hold only their own fields. One post is always exactly one or the other, never both, never neither.

"Why do Vote and Report point at Content instead of at ExamIntel or Materials directly?"

ELI5: A vote or a report needs to work on either type of post. Pointing both at the shared parent (Content) means one clean connection instead of two connections that are half the time empty.

---

4 Procedures, Triggers & Transactions

"Why use a trigger or procedure instead of just writing the rule in the app code?"

ELI5: App code can be skipped — a bug, a second app, someone hitting the database directly. A rule inside the database can't be bypassed no matter who's writing the data.

If pushed further: Example: `trg_enforce_siit_domain` re-checks the email domain even though the app already checks it client-side. That's on purpose — it's the difference between "please don't" and "physically can't."

"Why does sp_submit_content need to be a transaction?"

ELI5: Submitting one post is really two inserts — one into Content, one into its subtype. If the server crashed between those two inserts, you'd get a broken half-post with no actual content. A transaction says: do both, or undo both, never leave it half-done.

"Why JSON columns for format/topics instead of more tables?"

ELI5: A topic list ("recursion, sorting") is open-ended — could be one topic or ten. A whole separate table with joins is overkill for what it's used for here. JSON stores the list directly in the row, and it's the same idea the course itself introduces in Week 12.

"Why ENUM for content_type / exam_type?"

ELI5: These only ever have a few fixed, known options — there's no scenario where content_type is anything other than "exam_intel" or "materials." ENUM tells the database "only allow these values," so a typo gets rejected automatically instead of silently corrupting data.

"Why soft-delete instead of actually deleting reported content?"

ELI5: A real delete is gone forever, no undo, no record. Soft-delete (`removed_at`, `removed_by`) lets a moderator restore a wrongly-removed post, and leaves a paper trail of who took it down and why — important for something a student might dispute.